

Parallel and Distributed Programming

Parallel Loops

20/06/2026

Table des matières

Loops Can Hide Parallelism	1
Pointer-Chasing Loop	1
Complete Java Code: Sequential and Parallel Versions.....	2
Sequential version	2
Parallel/asynchronous version	2
The Parallel Loop	4
Sequential version using normal for	5
Parallel version using <code>IntStream.range(...).parallel()</code>	5

Loops Can Hide Parallelism

Many programs spend a lot of time inside loops. A loop repeats the same type of work many times. If each repetition is independent from the others, the loop can be transformed into a **parallel loop**

The simple question is:

Can iteration i run without waiting for iteration $i+1$?

If the answer is yes, the loop is a good candidate for parallel execution.

Pointer-Chasing Loop

Imagine we have students stored in a linked list:

head → Student 1 → Student 2 → Student 3 → ... → null

Each student has three marks: Math, Physics, and Programming. The task `compute(p)` calculates the student average and level. The traversal `p = p.next` is sequential because we must follow the linked list node by node. But the computation for each student is independent, so it can be executed asynchronously.

Complete Java Code: Sequential and Parallel Versions

PointerChasingSequentialParallel.java

Sequential version

In the sequential version, the program does everything **one by one**.

It starts from head, takes the first student, calculates the result, prints it, then moves to the next student. It does not start Student 2 before finishing Student 1.

Parallel/asynchronous version

In the parallel/asynchronous version, the program still follows the linked list **one node at a time** because `p.next` must be read sequentially.

But when the program reaches a node, it does not compute it immediately in the main thread. Instead, it sends the computation to an executor:

Point	Sequential version	Parallel / asynchronous version	What it means simply	Example in our scenario
Traversal of linked list	Sequential	Sequential	In both versions, the program must follow the linked list node by node using <code>p = p.next</code> .	The program visits Student 1, then Student 2, then Student 3.
Computation <code>compute(p)</code>	Done immediately	Sent to worker threads	In the sequential version, the main thread computes the result itself. In the parallel version, the main thread gives the	Student average is calculated directly in sequential, but submitted to <code>ExecutorService</code> in parallel.

computation to another thread.

Waiting	After every student	Mostly at the end	Sequential waits after each computation. Parallel launches many computations first, then waits later using <code>future.get()</code> .	Sequential: compute Student 1 and wait. Parallel: submit Student 1, Student 2, Student 3, then collect results.
Speed	Slower for heavy work	Faster if computation is heavy	If <code>compute(p)</code> takes time, parallel execution can save time because several students are processed at the same time.	If each student computation takes 500 ms, 4 worker threads can process several students together.
Complexity	Simple	More complex because of Future and threads	Sequential code is easier to read. Parallel code needs <code>ExecutorService</code> , <code>Future</code> , exception handling, and shutdown.	Sequential uses only a for loop. Parallel uses <code>submit()</code> , <code>Future</code> , <code>get()</code> , and <code>shutdown()</code> .
Main thread role	Does traversal and computation	Does traversal and task submission	In sequential, the main thread does everything. In parallel, the main thread mainly distributes work.	Main thread becomes like a teacher distributing copies to assistants.
Worker threads	Not used	Used	Sequential uses only one execution path. Parallel uses worker threads from a thread pool.	<code>Executors.newFixedThreadPool(4)</code> creates 4 workers.

Result collection	Result is available immediately	Result is collected later	In sequential, each result is ready after compute(p). In parallel, each result is inside a Future.	StudentResult result = future.get();
Best use case	Small work or simple programs	Heavy independent computations	Sequential is good when tasks are small. Parallel is useful when each node has expensive independent work.	Heavy score analysis, image processing, file analysis, data cleaning.
Main limitation	Does not use multiple cores	Traversal remains sequential	Parallel version does not make the linked-list traversal parallel; only the computation is parallel.	p = p.next still happens one step after another.

The Parallel Loop

CountedLoopSequentialParallel.java

The idea is based on this loop:

Imagine a university has **1,000,000 students**.

For each student:

$B[i]$ = score in Exam 1

$C[i]$ = score in Exam 2

$A[i]$ = total score = $B[i] + C[i]$

$D[i]$ = difference = $B[i] - C[i]$

Example:

Student 0:

$B[0] = 12$

$C[0] = 15$

$A[0] = 27$

$D[0] = -3$

Each student has a different index i . So student 0 writes only to $A[0]$ and $D[0]$, student 1 writes only to $A[1]$ and $D[1]$, and so on. This means the iterations are independent.

Sequential version using normal for

In the sequential version, the program uses a normal counted loop. It starts with $i = 0$, then $i = 1$, then $i = 2$, until the last student. For each student, it calculates the total score $A[i] = B[i] + C[i]$ and the difference $D[i] = B[i] - C[i]$. Only one task is executed at a time.

This version is simple and easy to understand. However, if the number of students is very large, for example 1,000,000 students, the program may take more time because all iterations are processed one after another.

Parallel version using `IntStream.range(...).parallel()`

In the parallel version, the program uses:

```
IntStream.range(0, B.length)
    .parallel()
    .forEach(i -> {
        A[i] = B[i] + C[i];
        D[i] = B[i] - C[i];
    });
```

Here, Java divides the indexes between several threads. This means that many students can be processed at the same time. parallel programming depends on decomposing a problem into parallel processes and mapping work to processing units. Java's `IntStream` supports both sequential and parallel operations, and Oracle explains that streams can execute sequentially or in parallel using `parallel()`

Point	Sequential Version	Parallel Version
Type of loop	Normal counted loop	Parallel counted loop
Execution	One iteration after another	Many iterations at the same time
Example	Student 0, then Student 1, then Student 2	Student 0, Student 1, Student 2 can be processed together
Java code	<code>for (int i = 0; i < n; i++)</code>	<code>IntStream.range(0, n).parallel()</code>
Speed	Slower for large data	Faster if the work is large enough
Simplicity	Very simple	A little more advanced
Threads	Mainly one thread	Several threads
Best use	Small data or simple work	Large data with independent iterations
Risk	Low complexity	Must avoid shared-variable conflicts

**In this
example**

Calculates scores one by one

Calculates scores for many students in parallel

Summary

Loops are important in parallel programming because many programs repeat the same work many times. If each iteration is independent, the loop can be transformed into a parallel loop. The key question is: **Can iteration i run without waiting for iteration $i+1$?** If yes, the loop is a good candidate for parallel execution.

The safest rule is: **parallelize a loop only when each iteration can run independently without corrupting shared data.**